

COMPUTEBENCH

Troubleshooting & Development Notes

A record of the real errors, debugging steps, fixes, and lessons learned while building the GPU benchmarking platform.

Project stack

Frontend	React + Vite + Recharts
Backend	Python + FastAPI
GPU Compute	NVIDIA CUDA
Telemetry	NVIDIA GPU monitoring / NVML
Database	SQLite / benchmark history
AI	Google Gemini API

Goal: Keep this as a personal interview/debugging reference. If something breaks again, search this PDF first.

1. What We Built

ComputeBench runs CUDA matrix-multiplication workloads, measures execution time and GFLOPS, records statistics such as median and standard deviation, reads live GPU telemetry, stores benchmark history, compares GPU sessions, and asks Gemini to interpret measured results.

The important architecture is:

```
React / Vite
↓ HTTP / JSON
FastAPI
↓
CUDA benchmark + GPU telemetry
↓
SQLite history
↓
GPU comparison JSON
↓
Gemini structured analysis
```

Core lesson: the GPU benchmark is the source of truth. Gemini explains the measurements; it does not perform or replace the benchmark.

2. Error & Problem Log

1. Backend had no virtual environment

Problem: The backend folder did not have its own venv, so Python packages and commands could become mixed with the system Python installation.

Why it happened: The project was being developed as a Python/FastAPI application and needed an isolated environment.

Solution: Create/use a project-level .venv and activate it before installing or running backend dependencies.

```
.\.venv\Scripts\Activate.ps1
```

2. PowerShell said the activation command was not recognized

Problem: The venv activation command was reported as an unrecognized cmdlet/command.

Why it happened: The terminal was either not in the project directory or the PowerShell activation path was wrong.

Solution: Navigate to the ComputeBench root and use the PowerShell activation script.

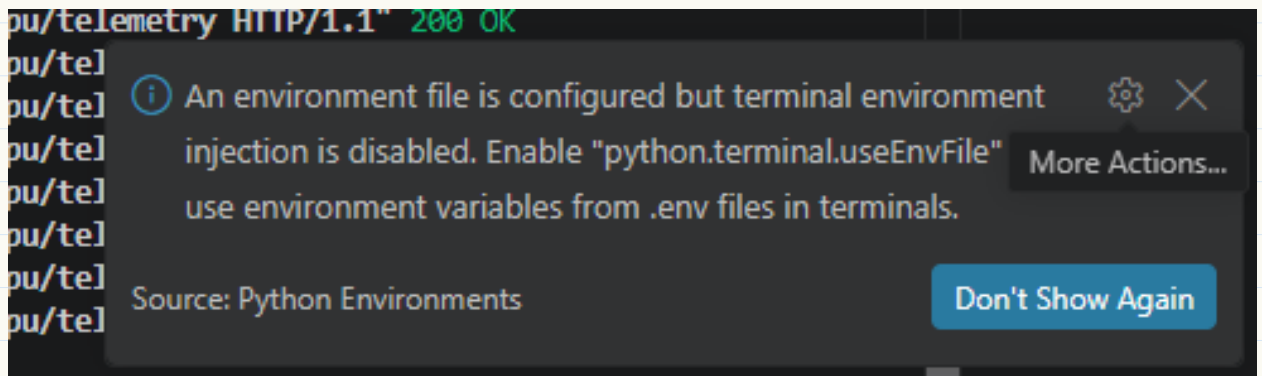
```
cd C:\Users\...\computebench\n\.venv\Scripts\Activate.ps1
```

3. VS Code .env terminal warning

Problem: VS Code displayed a message saying an environment file was configured and offered to enable `python.terminal.useEnvFile`.

Why it happened: VS Code detected the project's `.env` and was offering automatic environment-variable loading for Python terminals.

Solution: Enable the setting if desired, restart the terminal, and still use `python-dotenv` in application code so the app loads `.env` explicitly.



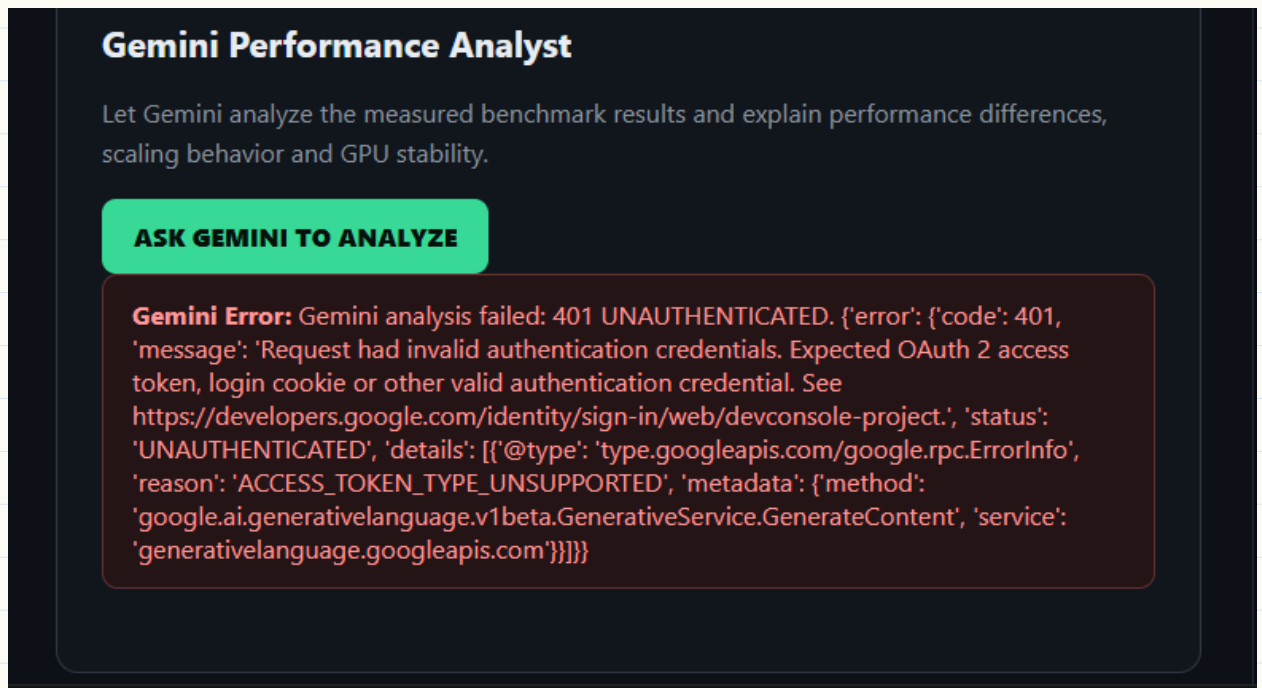
Screenshot: VS Code environment-file notification.

4. Gemini 401 UNAUTHENTICATED / AQ credential

Problem: The Gemini analysis endpoint returned 401 UNAUTHENTICATED with `ACCESS_TOKEN_TYPE_UNSUPPORTED`.

Why it happened: The initial Gemini credential/authentication path did not match the SDK/API call being used.

Solution: A new AI Studio key was created and the integration was moved to the authentication/API path that worked with the available credential.



Screenshot: Gemini 401 UNAUTHENTICATED error.

5. Gemini 2.5 Flash unavailable to new users

Problem: The account reported that Gemini 2.5 Flash was no longer available to new users.

Why it happened: Model availability depends on the project/account and current Google model access.

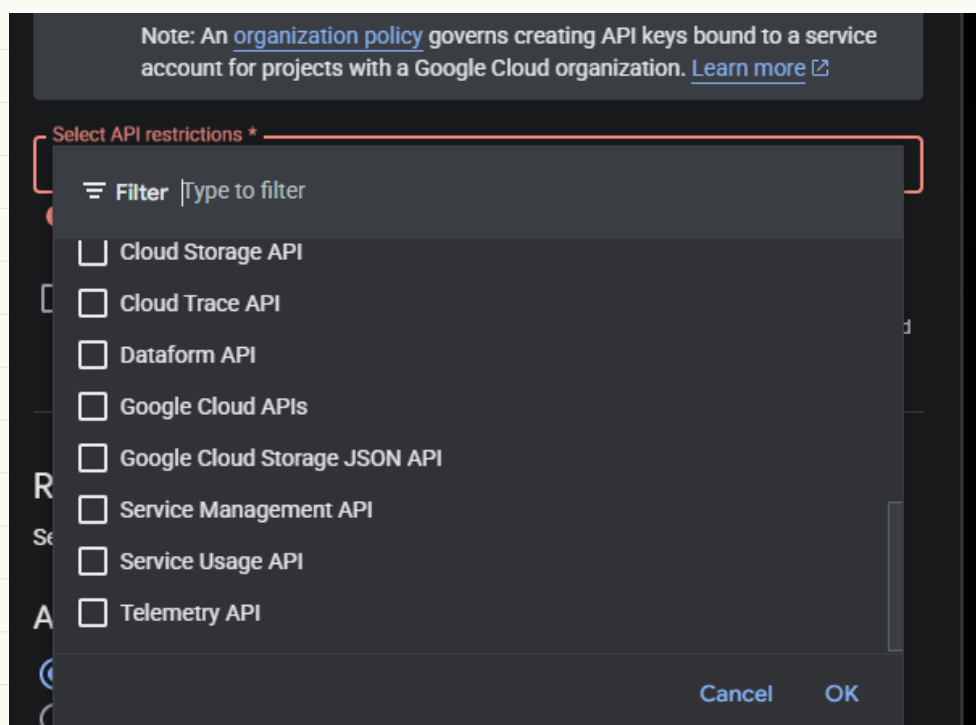
Solution: Stop hard-coding a model just because it is common. Query the models exposed by the API key and test an actually available model.

6. Generative Language API was not visible in the restriction picker

Problem: While creating a Cloud API key, Generative Language API could not be found in the API-restriction dropdown.

Why it happened: The Cloud Console credential screen and AI Studio's Gemini-key flow do not expose exactly the same choices.

Solution: Use the Gemini/AI Studio key flow for the Gemini credential rather than trying to force an unrelated Cloud API restriction setup.



Screenshot: Google Cloud credential restriction picker.

7. gemini-3.6-flash returned 503 UNAVAILABLE

Problem: A direct test reached Gemini but returned 503: the model was temporarily experiencing high demand.

Why it happened: The request and authentication worked; the selected model service was temporarily unavailable.

Solution: Check the models available to the key and use an available model rather than treating a 503 as an application-code failure.

```
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench> python -c "from google import genai;
import os; from dotenv import load_dotenv; load_dotenv(); client=genai.Client(api_key=os.getenv('GE
MINI_API_KEY')); [print(m.name, getattr(m,'supported_actions',None)) for m in client.models.list()]"
models/imagen-4.0-ultra-generate-001 ['predict']
models/imagen-4.0-fast-generate-001 ['predict']
models/veo-3.1-generate-preview ['predictLongRunning']
models/veo-3.1-fast-generate-preview ['predictLongRunning']
models/veo-3.1-lite-generate-preview ['predictLongRunning']
models/gemini-2.5-flash-native-audio-latest ['countTokens', 'bidiGenerateContent']
models/gemini-2.5-flash-native-audio-preview-09-2025 ['countTokens', 'bidiGenerateContent']
models/gemini-2.5-flash-native-audio-preview-12-2025 ['countTokens', 'bidiGenerateContent']
models/gemini-3.1-flash-live-preview ['bidiGenerateContent']
models/gemini-robotics-er-2-streaming-preview ['bidiGenerateContent']
models/gemini-3.5-live-translate-preview ['bidiGenerateContent']
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench>
```

Screenshot: Model listing / supported-actions debugging.

8. gemini-3.1-flash-generate-preview returned 404

Problem: The model appeared in models.list(), but generate_content() returned 404 and said the model was not supported for generateContent.

Why it happened: A model being listed does not guarantee that the specific API method/version used by the SDK supports it.

Solution: Inspect supported actions for each model and stop guessing model names.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench> python -c "from google import genai;
import os; from dotenv import load_dotenv; load_dotenv(); client=genai.Client(api_key=os.getenv('GE
MINI_API_KEY')); [print(m.name) for m in client.models.list()]"
models/imagen-4.0-ultra-generate-001
models/imagen-4.0-fast-generate-001
models/veo-3.1-generate-preview
models/veo-3.1-fast-generate-preview
models/veo-3.1-lite-generate-preview
models/gemini-2.5-flash-native-audio-latest
models/gemini-2.5-flash-native-audio-preview-09-2025
models/gemini-2.5-flash-native-audio-preview-12-2025
models/gemini-3.1-flash-live-preview
models/gemini-robotics-er-2-streaming-preview
models/gemini-3.5-live-translate-preview
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench>
```

Screenshot: Available-model inspection.

3. Gemini Debugging Timeline

The Gemini integration went through several distinct stages. This sequence is worth remembering because it demonstrates proper debugging rather than random code changes.

Stage A — Authentication failure

401 UNAUTHENTICATED → credential/authentication path was wrong.

Stage B — Authentication fixed

The new key loaded successfully and requests reached Gemini.

Stage C — Model availability problems

503 high demand, then 404 unsupported generateContent models.

Stage D — Capability inspection

models.list() and supported_actions were used instead of guessing.

Stage E — Working API path

The Interactions API successfully generated a response.

Stage F — Response extraction bug

The request worked, but output_text was not being read.

Stage G — Presentation problem

Markdown was visible in the UI, so Gemini output was changed to structured JSON and React became responsible for presentation.

This is a useful debugging pattern: authentication → availability → API capability → response parsing → UI rendering.

4. The Important Gemini Architecture

Do not let the AI become the UI. Gemini should return data; React should decide how that data looks.

```
Gemini
↓
{ overview, key_findings[], scaling_analysis, stability_analysis, performance_interpretation,
  recommendation, caveat }
↓
React cards / sections / badges
```

Why structured JSON?

- Predictable fields
- Easier validation
- Easier styling
- No raw Markdown in the UI
- Backend and frontend responsibilities stay separate

9. No normal text generateContent model appeared

Problem: The supported-actions output showed audio/live models with `generateContent` rather than normal text generation.

Why it happened: The API key/project exposed a restricted model set and the previous `generateContent` path was not appropriate for those models.

Solution: Move to the working Interactions API path and use a model supported by that path.

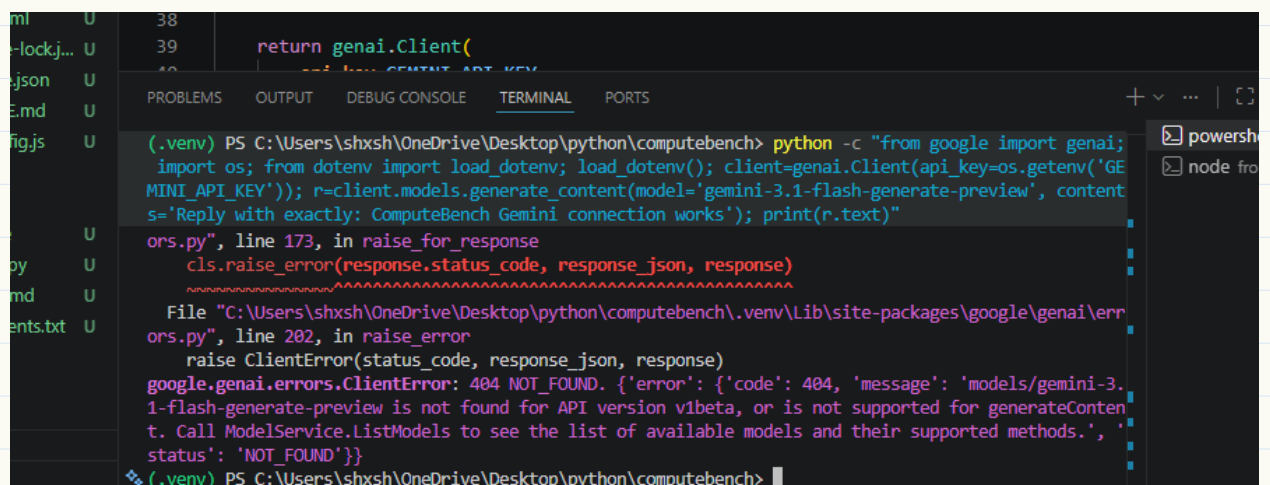
10. Gemini Interactions API worked but analysis was empty

Problem: The API request succeeded, but the application raised 'Gemini returned an empty response.'

Why it happened: The response text was being read from the wrong property.

Solution: Use `interaction.output_text` for the Interactions API response.

```
output_text = interaction.output_text
```



```
ml U 38
-lockj... U 39 return genai.Client(
json U 40
E.md U
figjs U
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench> python -c "from google import genai;
import os; from dotenv import load_dotenv; load_dotenv(); client=genai.Client(api_key=os.getenv('GE
MINI_API_KEY')); r=client.models.generate_content(model='gemini-3.1-flash-generate-preview', content
s='Reply with exactly: ComputeBench Gemini connection works'); print(r.text)"
ors.py", line 173, in raise_for_response
cls.raise_error(response.status_code, response_json, response)
File "C:\Users\shxsh\OneDrive\Desktop\python\computebench\.venv\Lib\site-packages\google\genai\err
ors.py", line 202, in raise_error
raise ClientError(status_code, response_json, response)
google.genai.errors.ClientError: 404 NOT_FOUND. {'error': {'code': 404, 'message': 'models/gemini-3.
1-flash-generate-preview is not found for API version v1beta, or is not supported for generateConten
t. Call ModelService.ListModels to see the list of available models and their supported methods.',
status': 'NOT_FOUND'}}
(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench>
```

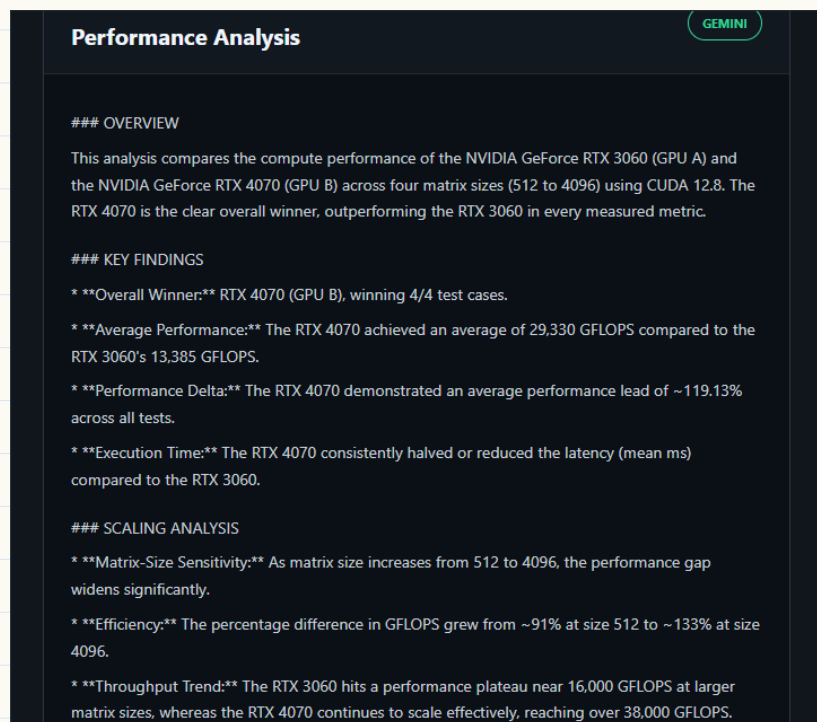
Screenshot: Gemini response/debugging output around the API call.

11. Gemini response looked like an HTML/Markdown document

Problem: The UI showed `### OVERVIEW`, **bold text**, bullets, and raw formatting.

Why it happened: Gemini was returning Markdown-like prose and React was displaying it as plain text.

Solution: Make Gemini return structured JSON and let React control the presentation.



Screenshot: Raw Markdown-style Gemini analysis visible in the UI.

12. Structured AI response needed validation

Problem: AI output can occasionally contain code fences or extra text around JSON.

Why it happened: LLM output is probabilistic and should be treated as untrusted/variable input.

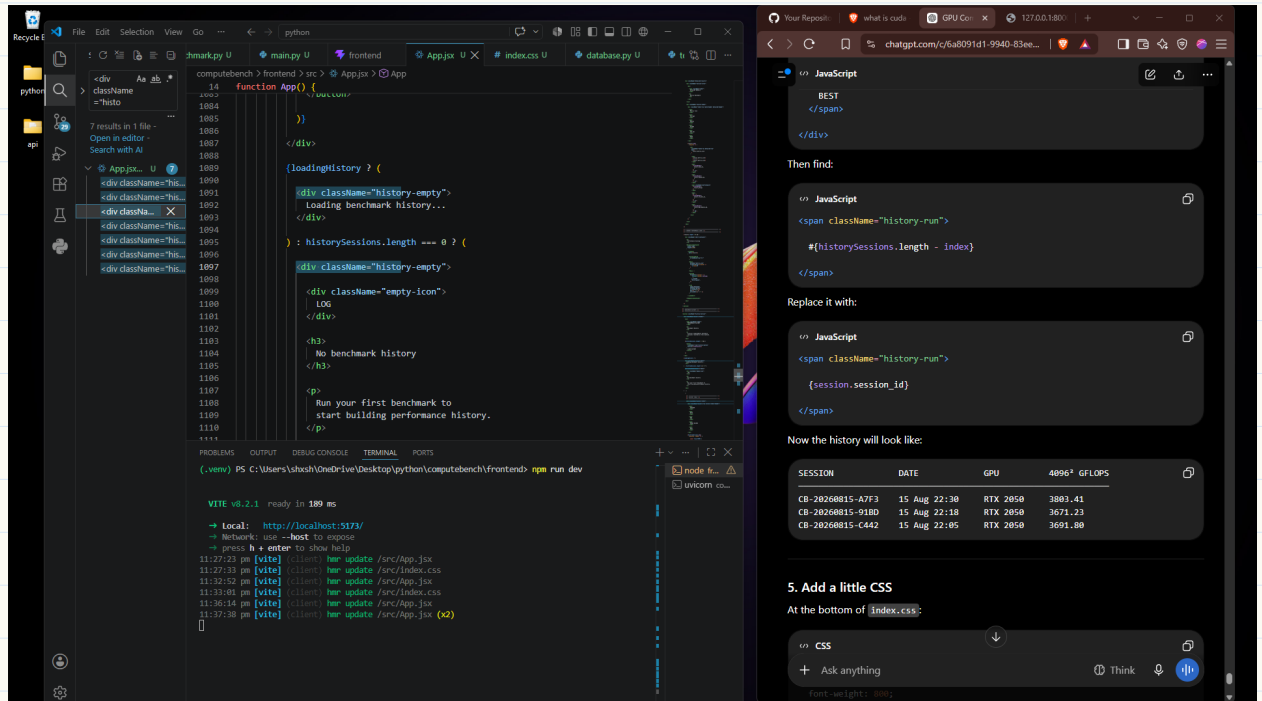
Solution: Strip code fences, extract the JSON object if needed, parse with `json.loads()`, validate required fields, and normalize `key_findings` to a list.

13. History initially appeared as raw/unstructured JSON

Problem: The history endpoint worked, but the response was not yet presented as a clean history table.

Why it happened: The backend was returning correct data before the frontend presentation layer was completed.

Solution: Map session data into a dedicated history table and display session ID, date, GPU and 4096² GFLOPS.



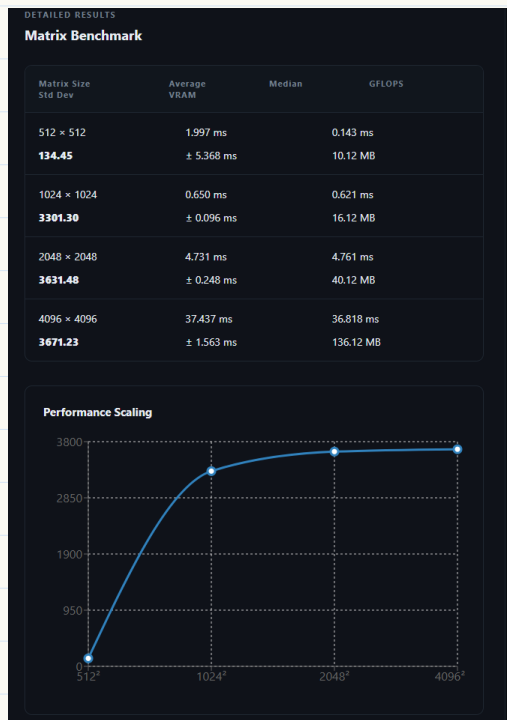
Screenshot: History UI while it was being structured.

14. Standard deviation / VRAM headings were missing

Problem: The detailed benchmark table contained values but the column headings were not correctly structured.

Why it happened: The React table header and result cells were out of sync.

Solution: Add explicit columns for Average, Median, GFLOPS, Std Dev and VRAM and ensure each row follows the same order.



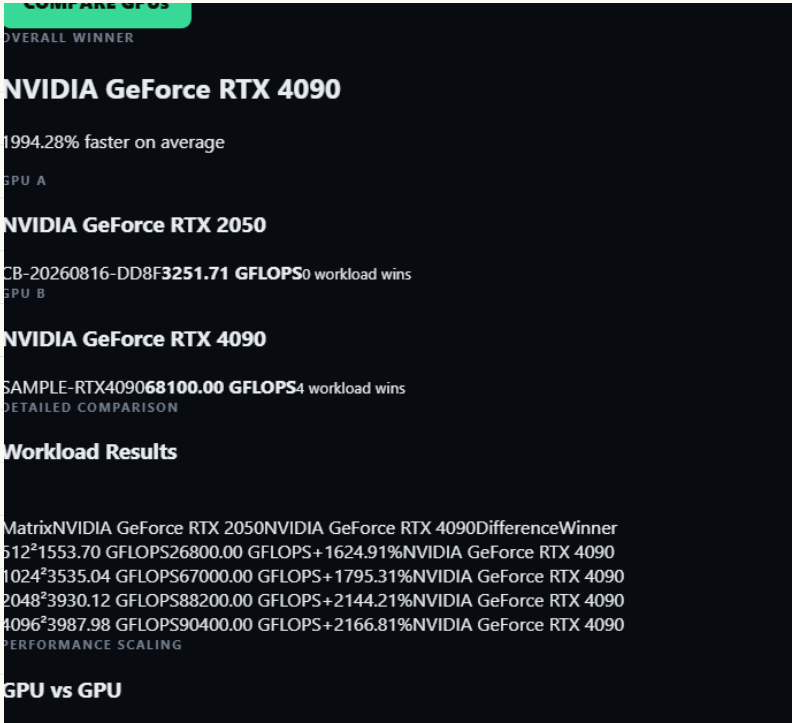
Screenshot: Detailed benchmark results with statistics.

15. GPU comparison UI was initially unstructured

Problem: Comparison output appeared compressed together instead of as a readable GPU-vs-GPU layout.

Why it happened: Raw comparison JSON was being displayed without a dedicated presentation model.

Solution: Create winner cards, per-matrix comparison rows, a comparison chart, and a separate AI analysis panel.



Screenshot: Early GPU comparison layout before it was polished.

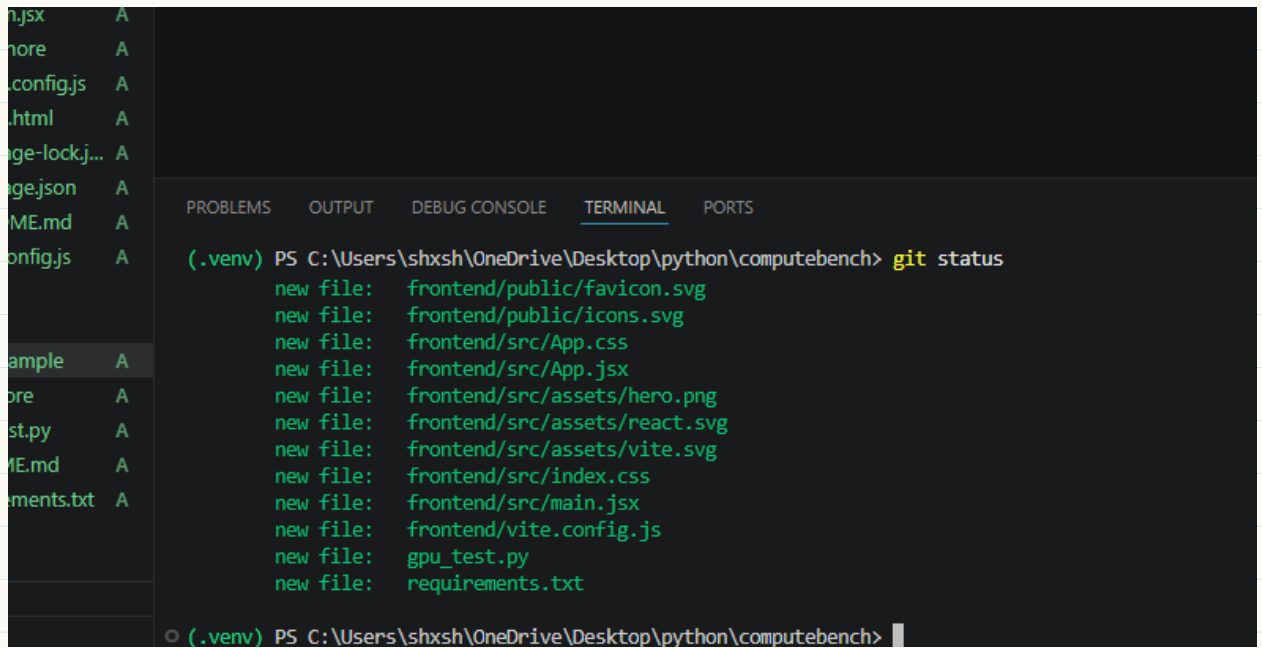
16. GitHub deployment/push safety

Problem: Before pushing, there was a risk of accidentally committing `.env`, the virtual environment, or `node_modules`.

Why it happened: Those files contain secrets, machine-specific dependencies, or huge generated folders.

Solution: Use `.gitignore`, stage with `git add .`, inspect `git status`, and verify only `.env.example` is tracked.

```
git ls-files | findstr \".env\"
```



```
h.jsx      A
nore       A
.config.js A
.html      A
ge-lock.j... A
ge.json    A
ME.md      A
onfig.js   A

ample      A
ore        A
st.py      A
ME.md      A
ments.txt  A

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench> git status
new file:   frontend/public/favicon.svg
new file:   frontend/public/icons.svg
new file:   frontend/src/App.css
new file:   frontend/src/App.jsx
new file:   frontend/src/assets/hero.png
new file:   frontend/src/assets/react.svg
new file:   frontend/src/assets/vite.svg
new file:   frontend/src/index.css
new file:   frontend/src/main.jsx
new file:   frontend/vite.config.js
new file:   gpu_test.py
new file:   requirements.txt

O (.venv) PS C:\Users\shxsh\OneDrive\Desktop\python\computebench>
```

Screenshot: Git staging status before the initial GitHub push.

5. Benchmark Problems & Concepts We Had to Get Right

GFLOPS: billion floating-point operations per second. For square matrix multiplication, the approximate operation count is $2 \times N^3$, so throughput can be estimated from the workload size and measured execution time.

Mean: average execution time across repeated runs.

Median: middle execution-time value after sorting. It is less sensitive to occasional outliers than the mean.

Standard deviation: measures run-to-run variation. Lower generally means more consistent measurements, but it does not mean the GPU is faster.

Warm-up runs: initial runs that help avoid measuring one-time startup effects as if they were normal steady-state performance.

VRAM: GPU memory used for matrices and other device allocations.

Scaling: how throughput changes as matrix size increases from 512^2 to 4096^2 .

Telemetry: live measurements such as utilization, temperature, VRAM usage and power.

Session: one benchmark run/group stored in history so two GPU runs can later be compared.

Important: a synthetic matrix benchmark is not the same thing as gaming FPS or overall real-world GPU performance. It measures a specific compute workload.

6. GitHub / Security Checklist

Before every push:

1. Check git status.
2. Make sure .env is ignored.
3. Make sure .venv and node_modules are ignored.
4. Confirm .env.example contains only a placeholder.
5. Commit meaningful changes.
6. Push to the main branch.

Safe example:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Unsafe example:

```
GEMINI_API_KEY=AI••••••••••••••••••••
```

Never put the real API key in README.md, App.jsx, main.py, screenshots, or Git history.

7. Interview Questions You Should Be Ready For

Question	Short answer
Why CUDA?	Because matrix multiplication is highly parallel and maps naturally to GPU threads.
Why GFLOPS?	It expresses compute throughput and lets us compare the amount of work completed per second.
Why repeat the benchmark?	A single run can be noisy. Repetition gives mean, median and standard deviation.
Why standard deviation?	To quantify run-to-run variability and judge consistency.
Why warm-up runs?	To reduce one-time initialization effects before measuring steady-state performance.
Why FastAPI?	It provides a lightweight Python API layer around CUDA, telemetry and database operations.
Why React?	It handles dynamic UI state, tables, charts and AI-result presentation.
What does Gemini do?	It interprets measured benchmark data. It does not create the benchmark measurements.
Why structured JSON from Gemini?	It gives the frontend predictable data instead of raw AI prose.
Why keep the API key in .env?	To avoid exposing secrets in source control.

8. One-Page Debugging Cheat Sheet

Backend won't start?

Check venv → activate → install requirements → run uvicorn.

PowerShell says activation not recognized?

Use `.\.venv\Scripts\Activate.ps1` from the project root.

Gemini 401?

Check the credential and authentication path. Verify the key loads from `.env`.

Gemini 503?

Authentication may already be working. Check whether the selected model is temporarily unavailable.

Gemini 404?

Do not assume a listed model supports your method. Inspect supported actions.

Gemini empty response?

Check the SDK response object. With Interactions API, use `interaction.output_text`.

AI looks like a webpage?

Return structured JSON and render it with React.

History looks like raw JSON?

Map backend data into a dedicated UI table.

Table headings wrong?

Ensure header order exactly matches row order.

GitHub push?

Ignore `.env`, `.venv` and `node_modules`. Commit `.env.example` instead.

Final architecture to remember:

CUDA benchmark → measurements → FastAPI → database/comparison → Gemini analysis → structured JSON → React UI

This document records the real development path: errors were not failures to hide; they are evidence of how the system was debugged and improved.